Back To Practice

Q1

Save

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int m, n, i, j;

    printf("Enter number of memory blocks:\n");
    scanf("%d", &m);

    int block[m], original[m];

    printf("Enter size of each block:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &block[i]);
        original[i] = block[i];
    }

    printf("Enter number of processes:\n");
    scanf("%d", &n);
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Back To Practice

Q1

Save

Completed

First Fit Contiguous Memory Allocation

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

Select Language : C (GCC 9.2.0)

```c
    int process[n];

    printf("Enter size of each process:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &process[i]);
    }

    for(i = 0; i < n; i++) {
        int found = 0;

        for(j = 0; j < m; j++) {
            if(block[j] >= process[i]) {

                int fragment = block[j] - process[i];

                printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                       i + 1, process[i], j + 1, original[j], fragment);

                block[j] -= process[i]; // allow reuse
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Back To Practice

Q1

Save

First Fit Contiguous Memory Allocation

Completed

Write a C program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

1. The first line contains an integer m representing the number of memory blocks.
2. The next line contains m integers representing the sizes of each memory block.
3. The next line contains an integer n representing the number of processes.
4. The next line contains n integers representing the sizes of each process.

Output Format

- For each process, the program prints whether the process is

```c
            printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
                   i + 1, process[i], j + 1, original[j], fragment);

            block[j] -= process[i]; // allow reuse
            found = 1;
            break;
        }
    }

    if(found == 0) {
        printf("Process %d of size %d is not allocated\n", i + 1, process[i]);
    }
}

return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2

38°C Search ENG 03:14 PM